PROJECT PROPOSAL

Autonomous Mobile Manipulation System

An End-to-End ROS 2 Robotics Project — Foundational Framework

Prepared by Kevin

August 2026

Project Overview

Develop an end-to-end ROS 2 robotic system, in simulation, that autonomously maps an environment, localizes itself, navigates to a specified object, detects and estimates the object's pose, plans a collision-free grasp, manipulates the object, transports it to a goal location, and reports mission state through visualization. The project is structured as ten sequential phases (Phase 0 through Phase 9), each with its own objective, core concepts, concrete tasks, deliverables, common pitfalls, and interview relevance, so that this document can serve as the single reference framework for the project from first commit to final release.

Why This Project

This project was chosen deliberately for breadth: it is one of the few single projects that spans localization and mapping, motion planning, computer vision, manipulation and inverse kinematics, and systems-level orchestration — the core competencies probed across the robotics industry broadly, from autonomous-vehicle companies to industrial and humanoid robotics. Phase 8 in particular, deliberate fault injection and recovery, is what separates this project from a tutorial walkthrough: most comparable personal projects stop at 'it worked once on camera.'

Final Demonstration Criteria

- Start in an unknown environment.
- Generate or load a map.
- Receive a high-level task (e.g., 'Move the red mug to the table').
- Navigate autonomously while avoiding obstacles.
- Detect and localize the target object.
- Plan and execute a grasp.
- Transport the object to a destination.
- Recover gracefully from at least one injected failure.
- Visualize robot state, map, detections, and planned path in RViz throughout.

Guiding Principles Throughout

- Interfaces are defined before implementation, and changed deliberately, not accidentally.
- Every phase produces a verifiable deliverable, not just 'code that runs once.'
- Metrics are logged from Phase 3 onward so that Phase 8's robustness improvements can be quantified, not just asserted.
- Documentation is written alongside each phase, not entirely deferred to Phase 9.

Repository Structure

The repository is organized by subsystem, mirroring the architecture defined in Phase 0. Each top-level directory corresponds to one or more ROS 2 packages.

Directory	Contents
/robot_description	URDF/Xacro, meshes, robot configuration files
/simulation	Gazebo worlds, simulation plugins, spawn/launch configs
/navigation	Nav2 parameter files, SLAM configuration, map files
/perception	Camera drivers, OpenCV utilities, detector node(s)
/manipulation	MoveIt 2 configuration, grasp planner
/mission_manager	Behavior tree / state machine implementation
/interfaces	Custom ROS 2 messages, services, and actions
/docs	Architecture diagrams, design notes, technical report
/tests	Integration scenarios, fault-injection test suite

PHASE 0

Environment & Architecture

Establish the technical foundation and system design before any implementation begins.

Objective

Set up a reproducible development environment and produce a complete architectural design of the system — every interface, message type, and package boundary decided on paper before a single line of ROS 2 code beyond scaffolding is written.

Core Concepts to Learn

- Software architecture for robotics: package decomposition, separation of concerns (perception vs. planning vs. control vs. execution)
- ROS 2 core concepts: nodes, topics, services, actions, parameters, launch files, and the discovery/DDS middleware layer underneath them
- Version control workflows for multi-module robotics repos (branching strategy, commit hygiene, .gitignore for build artifacts)
- Interface-first design: defining custom message/service/action types before implementation, and why this discipline prevents integration pain later

Concrete Tasks

- Install ROS 2 (Humble or Jazzy — pick one and note the rationale, e.g., LTS support window), Gazebo Harmonic, RViz2, and MoveIt 2, and verify each with a minimal smoke test (turtlesim, a Gazebo empty-world launch, an RViz session)
- Create the Git repository with a defined branching strategy (e.g., main protected, feature branches per phase, PR-based merges even as a solo developer, to build the habit)
- Draw a complete system architecture diagram showing every node, the topics/services/actions connecting them, and data flow direction, before writing implementation code
- Define the ROS package layout matching the repository structure below, with a stub package.xml and CMakeLists.txt/setup.py for each package
- Define all custom interfaces in /interfaces (e.g., DetectedObject.msg, GraspTarget.action, MissionStatus.msg) with explicit field types and units documented in comments
- Create a single top-level launch file that can start every subsystem (even as empty/stub nodes) to validate the launch architecture end-to-end from day one

Deliverables / Checkpoint

- Architecture diagram (exported to /docs, e.g., draw.io or Mermaid source plus rendered PNG/SVG)
- All ROS packages scaffolded and building cleanly with colcon build
- Interface definitions committed and compiling (ros2 interface show confirms each type)
- One-command top-level launch file that starts all stub nodes without errors

Common Pitfalls

- Skipping the architecture diagram and 'designing in code' — this is the single most common cause of painful refactors in later phases when two subsystems disagree on a message contract
- Under-scoping custom interfaces early, then having to break API compatibility across multiple packages simultaneously once perception and manipulation are both built
- Choosing a ROS 2 distribution without checking Gazebo/MoveIt 2 compatibility for that specific version

Interview Relevance

- Explain trade-offs between topics, services, and actions — and be ready to justify why you chose an action (not a service) for anything long-running and preemptible, like navigation or grasping
- Speak to why interface-first design matters in distributed robotic systems and how it mirrors API design in larger engineering organizations

PHASE 1

Robot Modeling

Build a physically accurate, correctly-articulated digital twin of the robot.

Objective

Produce a URDF/Xacro model of a differential-drive mobile base with a mounted 6-DOF arm that is kinematically correct, has plausible inertial properties, and renders and transforms correctly in RViz.

Core Concepts to Learn

- Kinematic chains, links, and joints; the distinction between fixed, revolute, prismatic, and continuous joints
- Coordinate frames and the TF2 tree: parent/child relationships, static vs. dynamic transforms
- Rigid body inertial properties (mass, center of mass, inertia tensor) and why inaccurate inertias destabilize physics simulation even when visuals look correct
- Xacro macros for parametric, reusable robot descriptions instead of duplicated URDF blocks
- Forward kinematics as the mathematical basis for everything downstream (TF, MoveIt, visualization) depending on this model being correct

Concrete Tasks

- Select a differential-drive base (two driven wheels + casters) and a 6-DOF arm design (or adapt an open-source arm model such as a simplified UR-style or custom design) appropriate for tabletop pick-and-place
- Author the URDF/Xacro: links with correct visual, collision, and inertial elements; joints with realistic limits, velocity, and effort bounds
- Build and verify the TF tree using ros2 run tf2_tools view_frames, checking there are no disconnected frames or duplicate parents
- Load the model in RViz with joint_state_publisher_gui to manually sweep every joint and visually confirm no mesh interpenetration or broken pivots
- Write a unit-style check script that queries joint limits programmatically and asserts they match the design spec (sanity check against typos)

Deliverables / Checkpoint

- Complete URDF/Xacro model in /robot_description
- TF tree diagram (frames.pdf from view_frames) with no orphaned frames
- RViz screenshot/video demonstrating full range of motion without visual defects

Common Pitfalls

- Zero or near-zero inertia values that cause the robot to behave erratically or explode in simulation once Phase 2 begins
- Mismatched joint axis conventions between visual and collision geometry, causing the arm to look correct in RViz but collide incorrectly in physics

- Forgetting mimic joints or coupling constraints on grippers, which manipulation planning in Phase 5 will depend on

Interview Relevance

- Be able to whiteboard forward kinematics for a simple 2-3 DOF chain from memory — this is a very common robotics interview question
- Explain why inertial properties matter for simulation fidelity even when they're invisible in a static screenshot

PHASE 2

Simulation

Bring the robot to life in a physics-accurate virtual world with working sensors.

Objective

Spawn the modeled robot into a Gazebo world with realistic physics, integrate RGB-D camera and lidar sensors, and confirm that odometry, TF, and sensor topics are all mutually consistent.

Core Concepts to Learn

- Physics simulation fundamentals: solver timestep, contact dynamics, friction models, and why simulated dynamics diverge from reality (the sim-to-real gap)
- Sensor modeling: how simulated lidar (ray-casting) and RGB-D cameras (rendered depth buffers) approximate real sensor noise and field-of-view characteristics
- Odometry sources and drift: wheel encoder odometry vs. ground-truth simulation odometry, and why real robots need to fuse both with additional sensors
- The ROS 2 <-> Gazebo bridge (ros_gz or gazebo_ros_pkgs) and how simulated clock/time synchronization works with use_sim_time

Concrete Tasks

- Build a warehouse or apartment-style Gazebo world with distinguishable static obstacles and open floor space for navigation testing
- Spawn the robot via the bridge, confirming the URDF loads with correct physics properties inherited from Phase 1
- Attach and configure an RGB-D camera plugin and a 2D or 3D lidar plugin at sensible mounting poses on the robot
- Validate every sensor topic with ros2 topic hz and ros2 topic echo, confirming expected publish rates and non-degenerate data (no all-zero depth images, no NaN lidar ranges)
- Verify odometry-to-TF consistency: drive the robot manually via teleop and confirm the TF tree updates smoothly with no jumps or frame flapping
- Package launch files so the full simulation environment (world + robot + sensors) starts from one command with use_sim_time correctly propagated to every node

Deliverables / Checkpoint

- Gazebo world file(s) in /simulation
- Verified, documented sensor topic list with expected rates and frames
- Teleop-drivable robot in simulation with clean TF/odometry

Common Pitfalls

- Forgetting to set use_sim_time: true consistently across every node, causing subtle timestamp mismatches that silently break TF lookups later
- Sensor mount frames not matching the URDF's declared optical frame conventions (REP 103/104), which corrupts every downstream perception transform

- Physics instability from Phase 1 inertia issues only becoming visible now, under real contact forces

Interview Relevance

- Discuss the sim-to-real gap concretely: which parts of your simulated sensors/physics you know are unrealistic, and how you'd validate on real hardware
- Explain why timestamp and frame consistency are foundational to every higher-level robotics algorithm

PHASE 3

Navigation

Give the robot the ability to map, localize, and move through the world autonomously.

Objective

Achieve reliable autonomous waypoint navigation with dynamic obstacle avoidance using Nav2, backed by either live SLAM or a pre-built and saved occupancy map.

Core Concepts to Learn

- SLAM (Simultaneous Localization and Mapping): the core chicken-and-egg problem of building a map while localizing within it, and how modern solvers (e.g., slam_toolbox's graph-based approach) resolve it
- Occupancy grid maps vs. costmaps: static map layer, inflation layer, and dynamic obstacle layer, and how Nav2 composes them
- Global vs. local planning: the division of labor between a global planner (e.g., NavFn/Smac) computing a coarse path and a local controller (e.g., DWB/MPPI) executing it reactively around dynamic obstacles
- Localization: AMCL (particle filter localization) fundamentals — motion model, sensor model, and particle resampling
- Costmap tuning: inflation radius, cost scaling factor, and the trade-off between safety margin and passable corridor width

Concrete Tasks

- Run SLAM (slam_toolbox) while teleoperating the robot through the full simulated environment to build an occupancy map; save it
- Alternatively/additionally, configure AMCL to localize against the saved map on subsequent runs, and compare startup behavior against live SLAM
- Configure the Nav2 stack: global planner, local controller, costmap layers, and recovery behaviors
- Tune costmap inflation and controller parameters iteratively, using RViz to visualize planned vs. executed paths
- Send waypoint navigation goals programmatically (not just via RViz clicks) using the Nav2 action client, to build the interface Phase 6 will call
- Introduce dynamic obstacles (spawned or moved mid-run) and verify the local controller replans around them without global replanning thrashing
- Define and log a navigation success-rate metric across a fixed set of test waypoints (e.g., 20 trials, record completion/failure and time-to-goal)

Deliverables / Checkpoint

- Saved occupancy map plus SLAM launch configuration
- Tuned Nav2 parameter file(s) in /navigation with inline comments explaining non-default values
- Logged navigation success-rate metrics across a repeatable test set
- A working Python/C++ node that sends navigation goals programmatically via the action interface

Common Pitfalls

- Over-tightening costmap inflation to 'fix' a collision, which then causes the planner to fail in narrow doorways — tuning is a trade-off, not a one-directional fix
- Treating global replanning frequency as free — excessive replanning near dynamic obstacles causes visible path oscillation ('planner thrashing')
- Not testing localization recovery from a bad initial pose estimate, which is a common real-world failure mode

Interview Relevance

- Be ready to explain the global/local planner division of labor and name concrete algorithms on each side (e.g., A*/Smac Planner globally, DWB or MPPI locally)
- Discuss costmap layer composition and how you'd debug a robot that clips corners versus one that's overly conservative
- This phase alone maps directly onto what AV and mobile-robotics interviewers probe most — be prepared to discuss it in depth

PHASE 4

Perception

Give the robot eyes: detect, localize, and publish real-world object poses.

Objective

Implement a perception pipeline that detects a target object in RGB imagery, estimates its 3D pose using depth data, and publishes that pose in the map frame for downstream consumption.

Core Concepts to Learn

- 2D object detection fundamentals: bounding-box detectors (YOLO family) vs. classic feature-based detection, and the accuracy/latency trade-off relevant to real-time robotics
- RGB-D sensor fusion: aligning color and depth pixel grids, and back-projecting a 2D detection into a 3D point using camera intrinsics
- Coordinate frame transforms in perception: camera-optical-frame -> base-link -> map, and why every detection is meaningless without a correctly-stamped TF chain
- Pose estimation vs. pose refinement: a simple centroid/bounding-box-based estimate versus more advanced approaches (e.g., ICP refinement against a known object model) and when the added complexity is justified
- Detection confidence and false-positive handling as a preview of the robustness work in Phase 8

Concrete Tasks

- Set up synchronized RGB and depth image subscription using message_filters for time-aligned pairs
- Integrate a YOLO (or comparable) detector, running either on CPU or GPU depending on available hardware, and benchmark its inference latency against the sensor publish rate
- Back-project detected bounding-box centers into 3D camera-frame coordinates using the depth image and published camera_info intrinsics
- Use tf2 to transform the detected 3D point from the camera frame into the map frame, handling transform timing/extrapolation correctly
- Publish detections on a custom DetectedObject.msg topic (class label, 3D pose, confidence, timestamp) as defined in Phase 0's interfaces
- Add an RViz visualization (e.g., interactive markers or a MarkerArray) showing detected object poses overlaid on the map and camera feed
- Stress-test detection against occlusion, poor lighting, and objects at the edge of the camera's field of view, logging failure cases for Phase 8

Deliverables / Checkpoint

- Working perception node(s) in /perception publishing DetectedObject messages
- RViz visualization of live detections with 3D pose markers
- A short benchmark note: detection latency, frame rate impact, and accuracy on a fixed test set of object placements

Common Pitfalls

- Using the wrong depth value at a bounding-box center when that pixel happens to be a hole/no-return in the depth image — averaging over a small patch is more robust than a single pixel sample
- TF timing mismatches: looking up a transform at the wrong timestamp (e.g., 'now' instead of the image's stamp) causes systematic pose offset that's easy to misdiagnose as a detector problem
- Not accounting for camera intrinsics distortion, which matters more at wide-angle lens edges

Interview Relevance

- Explain end-to-end how a 2D bounding box becomes a 3D pose in the map frame — this is a very common systems-design interview question in perception-adjacent robotics roles
- Discuss the precision/recall trade-off of your detector's confidence threshold and how you'd tune it for a safety-critical downstream action like grasping

PHASE 5

Manipulation

Turn a detected pose into a successful physical grasp.

Objective

Configure MoveIt 2 for the arm, generate feasible grasp poses from detected object geometry, plan and execute collision-free trajectories, and reliably pick up the target object.

Core Concepts to Learn

- Motion planning fundamentals: configuration space (C-space), sampling-based planners (RRT/RRT-Connect), and why planning happens in joint space rather than task space
- Inverse kinematics (IK): given a desired end-effector pose, solving for joint angles, including the reality of multiple solutions and unreachable poses
- The MoveIt 2 planning scene: how collision objects (the table, the object, the robot itself) are represented and kept in sync with perception output
- Grasp pose generation: simple heuristic approaches (top-down or axis-aligned grasps based on object bounding geometry) versus learned grasp-quality models, and why starting heuristic is the right scoping choice here
- Trajectory execution and controller interfaces: the handoff from a planned joint trajectory to actual actuator commands, and how execution failure is detected

Concrete Tasks

- Configure MoveIt 2 for the arm using the MoveIt Setup Assistant: planning groups, end-effector definition, and IK solver selection (e.g., KDL or a faster analytic/IK-fast solver if feasible)
- Populate the planning scene programmatically from Phase 4's detections and known static geometry (table, walls) so the planner treats them as real obstacles
- Implement a grasp pose generator that takes a detected object's pose/bounding box and proposes one or more candidate end-effector grasp poses with an approach vector
- Compute IK for candidate grasp poses and filter out unreachable ones before planning, to avoid wasting planner time on infeasible goals
- Plan and execute collision-free trajectories to the selected grasp pose, then a pre-grasp approach, then the grasp itself
- Implement gripper open/close control and verify grasp success via a simple check (e.g., gripper position feedback or a contact/force threshold if simulated)
- Plan and execute a place trajectory to a specified destination pose, then release

Deliverables / Checkpoint

- MoveIt 2 configuration package in /manipulation
- Grasp pose generation module with documented heuristic
- Demonstrated successful pick-and-place cycle in simulation, recorded as video evidence

Common Pitfalls

- Stale or missing planning-scene updates causing the planner to 'succeed' against an outdated world model and collide with an object that moved
- Choosing grasp approach vectors that are kinematically awkward for the arm's joint limits, causing IK failures that look like a planning bug but are really a grasp-generation bug
- Underestimating how much of this phase's time goes to debugging rather than writing new code — budget accordingly

Interview Relevance

- Be able to explain the difference between IK and motion planning, since candidates frequently conflate them
- Discuss why you chose a heuristic grasp generator over a learned one, and what a learned approach (e.g., a grasp-quality CNN) would add — shows awareness of the field beyond just what you built

PHASE 6

Mission Executive

Orchestrate every subsystem into one coherent, recoverable mission.

Objective

Implement a finite-state machine or behavior tree that sequences navigation, perception, and manipulation into a complete mission, with retry and timeout handling at every state.

Core Concepts to Learn

- Finite-state machines vs. behavior trees: what each is good at, and why behavior trees are increasingly preferred in robotics for their composability and reactivity
- Task-level orchestration versus low-level control: the mission executive should command subsystems through their existing action interfaces, not reimplement their logic
- Failure semantics: distinguishing a recoverable failure (retry with backoff) from a terminal one (abort mission, report state) at each stage
- Timeout design: choosing timeout durations that are long enough to tolerate legitimate variance (e.g., a longer nav goal) but short enough to catch real hangs

Concrete Tasks

- Choose and justify FSM vs. behavior tree (e.g., using py_trees or BehaviorTree.CPP) based on project complexity and your own learning goals
- Implement states/nodes: Idle -> Navigate -> Search -> Detect -> Align -> Grasp -> Transport -> Place -> Complete, each calling the relevant Phase 3-5 action interface
- Add a 'Search' behavior for when the target object isn't immediately visible (e.g., a scan rotation or a set of candidate viewpoints to navigate to)
- Add an 'Align' behavior that fine-adjusts robot pose relative to the detected object before the arm attempts a grasp, since navigation alone rarely leaves ideal grasp geometry
- Add retry logic with bounded attempts and backoff at each failure-prone state (detection, grasp)
- Add per-state timeout handling that transitions to a defined failure/recovery state rather than hanging indefinitely
- Log every mission event (state transitions, retries, failures, timings) to a structured log file for later analysis in Phase 8

Deliverables / Checkpoint

- Mission executive package in /mission_manager implementing the full state graph
- A high-level task interface accepting a command like 'move the red mug to the table' (even if parsed with simple structured input rather than full NLP)
- Structured mission event logs demonstrating multiple full mission runs

Common Pitfalls

- Letting the mission executive reach into subsystem internals instead of going through their public action interfaces — this recreates tight coupling that Phase 0's interface design was meant to prevent

- Infinite retry loops with no bound, which mask real failures instead of surfacing them
- Treating 'Align' as optional — skipping it is a common cause of Phase 5 grasp failures that look like manipulation bugs but are actually mission-sequencing bugs

Interview Relevance

- Be ready to compare FSMs and behavior trees concretely, with an example of a failure case each handles differently
- This phase is a strong answer to 'tell me about a time you designed a complex system with many failure modes' — prepare a STAR-format story from it

PHASE 7

System Integration

Prove the whole system works together, not just each part in isolation.

Objective

Launch the complete stack from a single command and verify that every subsystem dependency (perception -> manipulation, navigation -> manipulation) holds under repeated, stress-tested mission runs.

Core Concepts to Learn

- Integration testing in robotics: why unit-level correctness of each subsystem doesn't guarantee system-level correctness, and how timing/ordering bugs hide at the boundaries
- Launch file composition at scale: parameter files, namespacing, and remapping to avoid topic collisions as the system grows
- Repeatability and statistical evaluation: running the same mission many times and treating the result as a distribution, not a single pass/fail

Concrete Tasks

- Compose all per-subsystem launch files into one top-level launch file with consistent namespacing and parameter loading
- Verify explicitly that manipulation only begins after navigation reports goal success (no race conditions from optimistic sequencing)
- Verify that perception detections feeding manipulation are recent enough (timestamp-checked) and not stale from a previous mission
- Run repeated mission trials (e.g., 20-50 runs) back-to-back without manual intervention between runs, resetting simulation state programmatically
- Measure and record an end-to-end mission completion rate, plus a breakdown of which phase/state most often causes failure

Deliverables / Checkpoint

- Single top-level launch file starting the entire integrated system
- A test harness script that runs N missions automatically and aggregates results
- An end-to-end completion-rate report with per-stage failure breakdown

Common Pitfalls

- Manually resetting the simulation between test runs, which silently limits how many trials you actually run — automate this early
- Discovering in this phase that two subsystems' custom messages have subtly incompatible field semantics (e.g., is a pose in map frame or base_link frame) — a symptom of interface ambiguity from Phase 0

Interview Relevance

- This phase is your strongest answer to 'how do you approach debugging a complex distributed system' — you'll have concrete, specific failure stories, not hypotheticals

PHASE 8

Robustness

Deliberately break the system to find out how it fails, and make it fail gracefully.

Objective

Inject realistic failure conditions — sensor noise, moving obstacles, failed detections, failed grasps, blocked paths — and extend the system to detect and recover from each, then quantify the improvement.

Core Concepts to Learn

- Fault injection as a testing methodology: deliberately introducing failure modes rather than only waiting to discover them
- Graceful degradation: the distinction between a system that fails safely (stops, reports, waits for help) and one that fails silently or dangerously
- Noise modeling: how added sensor noise (Gaussian noise on lidar ranges, depth dropout, detector false negatives) approximates real-world sensor imperfection
- Recovery behavior design: re-planning, re-attempting detection from a different viewpoint, or aborting to a safe state as distinct recovery strategies suited to different failure types

Concrete Tasks

- Inject Gaussian noise into simulated lidar/depth data and re-run navigation and perception to measure degradation
- Add moving obstacles (e.g., scripted actors or dynamically-spawned objects) during navigation and manipulation phases
- Force detection failures (e.g., occlude the object, or artificially suppress detector output) and verify the mission executive's Search/retry logic actually engages correctly
- Force grasp failures (e.g., randomize grasp pose slightly beyond tolerance) and verify retry logic and eventual graceful abort with a reported failure state
- Simulate a blocked navigation path with no available route and verify the robot reports failure rather than hanging or endlessly replanning
- Record failure statistics before and after robustness improvements, so you can quantify the effect (e.g., mission completion rate under noise: X% before, Y% after)

Deliverables / Checkpoint

- A documented fault-injection test suite in /tests
- Before/after robustness metrics demonstrating measurable improvement
- Updated mission executive and subsystem code with the recovery behaviors this phase required

Common Pitfalls

- Only testing 'happy path plus one failure at a time' — realistic robustness testing should occasionally combine multiple simultaneous failure conditions
- Adding recovery logic that masks a failure so well that a real underlying bug (not just injected noise) goes unnoticed

Interview Relevance

- This is the single most differentiating phase for interviews — most candidates never build fault injection into a personal project. Prepare specific before/after numbers to cite
- Be ready to discuss the difference between fail-safe and fail-silent behavior, and give a concrete example from this phase of each

PHASE 9

Polish

Package the work so it's reproducible, understandable, and presentable.

Objective

Bring documentation, diagrams, and a recorded demonstration up to a standard where another developer could reproduce the project from the README alone, and where the work is immediately legible to a recruiter or interviewer.

Core Concepts to Learn

- Technical writing for reproducibility: what a README needs to contain (setup, dependencies, launch instructions, expected behavior) versus what belongs in deeper docs
- Communicating system design visually: keeping the architecture diagram synchronized with the as-built system, not the Phase 0 plan

Concrete Tasks

- Write complete documentation for every package, keeping the /docs architecture diagram updated to match final implementation (not the original Phase 0 sketch)
- Record a polished demonstration video (under 5 minutes) showing the full pipeline: unknown environment -> mapping -> task command -> navigation -> detection -> grasp -> transport -> place -> mission complete, including at least one recovered failure from Phase 8
- Write a technical report summarizing architecture, key design decisions, quantitative results (navigation success rate, mission completion rate, robustness before/after), and lessons learned
- Clean the repository: remove dead code and experimental branches, finalize the README, and ensure the one-command launch actually works from a fresh clone
- Tag a v1.0 release

Deliverables / Checkpoint

- Finalized README enabling another developer to reproduce the project
- Architecture diagram matching the as-built system
- Demo video under 5 minutes
- Written technical report
- Tagged v1.0 release on the repository

Common Pitfalls

- Treating documentation as an afterthought squeezed into the last day — budget real time for this phase, since it's what recruiters and interviewers actually see first
- A demo video that shows only the happy path — including one visible recovery from a Phase 8 failure is far more impressive than a flawless run

Interview Relevance

- Your README and report become your interview prep sheet — writing them well now saves re-deriving your own design rationale under interview pressure later

Definition of Done

- The entire system launches from a single command.
- All ROS packages are documented.
- The robot completes autonomous pick-and-place without manual intervention in the demonstration world.
- The architecture diagram matches the as-built implementation.
- The README enables another developer to reproduce the project from a fresh clone.
- A demo video under 5 minutes showcases the full pipeline, including at least one recovered failure.
- Quantitative results are recorded: navigation success rate, end-to-end mission completion rate, and robustness metrics before/after Phase 8.
- The repository is tagged as release v1.0.

Appendix: Suggested Tooling

Area	Suggested Tools
Middleware	ROS 2 (Humble or Jazzy LTS)
Simulation	Gazebo Harmonic, ros_gz bridge
Visualization	RViz2
SLAM / Localization	slam_toolbox, Nav2 AMCL
Navigation	Nav2 (planner, controller, behavior tree navigator)
Perception	OpenCV, YOLO (Ultralytics or equivalent), message_filters
Manipulation	MoveIt 2, KDL or IKFast IK solver
Mission Orchestration	py_trees or BehaviorTree.CPP, or a custom smach-style FSM
Testing	colcon test, custom fault-injection scripts, launch_testing

Appendix: Resume & Interview Framing

Suggested one-line resume framing once complete:

> *"Designed and built an end-to-end autonomous mobile manipulation system in ROS 2 / Gazebo — integrating SLAM, Nav2 path planning, a YOLO-based perception pipeline, and MoveIt 2 grasp planning behind a behavior-tree mission executive; achieved a [X]% end-to-end mission completion rate and demonstrated measurable robustness gains under injected sensor noise and dynamic obstacles."*

Fill in the bracketed metric once Phase 7/8 data is available — a specific number is significantly more credible to both recruiters and interviewers than a qualitative claim.

Mapping Phases to Common Interview Question Categories

Question Category	Phases to Draw From
System design / architecture	Phase 0, Phase 7
Kinematics & math fundamentals	Phase 1, Phase 5
Sensor fusion & simulation fidelity	Phase 2, Phase 4
Planning & control algorithms	Phase 3, Phase 5
Debugging distributed/complex systems	Phase 7, Phase 8
Behavioral: handling ambiguity / failure	Phase 6, Phase 8